

PostgreSQL Streaming Replication

How To Configure Streaming Replication in PostgreSQL

Table of Contents

- A. Environment
- 0. Pre-requisites

On Master (rac1)

- 1. Create Replication User
- 2. Update postgresql.conf
- 3. Update pg_hba.conf
- 4. Reload/Restart Configuration

On Standby_(rac2, rac3)

- 5. Stop PostgreSQL on Standbys
 - 5.1 Run from rac2(standby1)
 - 5.2 Run from rac3(standby2)
 - 6. Delete Files under Data Directory
 - 6.1 Run from rac2(standby1)
 - 6.2 Run from rac3(standby2)
 - 7. Run the backup on the standby to copy data from the primary
 - 7.1 Run from rac2(standby1)
 - 7.2 Run from rac3(standby2)
 - 8. Verify primary_conninfo on Standbys
 - 8.1 Run from rac2(standby1)
 - 8.2 Run from rac3(standby2)
 - 9. Start PostgreSQL on Standbys
 - 9.1 Run from rac2(standby1)
 - 9.2 Run from rac3(standby2)
 - 10. Verify Streaming Replication
-

A. Environment

HOSTNAME	IP	OS	VERSION	ROLE	HOT STANDBY	REPLICATION	SYNC TYPE
rac1	192.168.2.21	Linux	PostgreSQL L 17	Primary			
rac2	192.168.2.22	Linux	PostgreSQL L 17	Standby	Yes (Active DG)	Streaming	async (Maximum Performance)
rac3	192.168.3.20	Linux	PostgreSQL L 17	Standby	Yes (Active DG)	Streaming	async (Maximum Performance)

o. Pre-requisites

1. Install PostgreSQL on all servers (rac1, rac2, rac3)
2. Ensure network connectivity (port 5432) between nodes
3. Ensure same PostgreSQL version and paths on all nodes

Overview:

Physical streaming replication works at the cluster level, not at the database level in PostgreSQL.

Physical (streaming) replication = whole cluster

Logical replication = per database or table

Streaming replication is a way to keep one or more standby servers in sync with a primary server in real-time.

It ensures that all changes made on the primary database are continuously sent to the standby servers. This provides high availability and read scaling.

How It Works:

1. Primary records changes

Every change (INSERT, UPDATE, DELETE) is first written to the WAL (Write-Ahead Log) on the primary server.

2. Standbys connect to the primary

Each standby uses its connection info (primary_conninfo) to connect to the primary.

```
[postgres@rac2 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.21 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'rac2_standby_slot'
[postgres@rac2 ~]$
```

3. Primary sends WAL to standbys

Once the connection is established, the primary streams the WAL changes to the standbys continuously.

4. Standbys apply changes

Standbys receive WAL and replay it to stay in sync with the primary. Hot standbys can even allow read-only queries while receiving data.

5. Replication modes

Asynchronous: Primary doesn't wait for standby, standby may lag slightly. (Maximum Performance)

Synchronous: Primary waits for standby acknowledgment before committing. zero data loss. (Maximum Protection).

6. Important Notes:

Asynchronous = Maximum Performance (Oracle) - Safe

Replication slots retain WAL files, so if a standby goes offline for a long time, disk usage can grow on the primary. Monitor pg_wal size.

If a standby is permanently removed, drop the slot to free space.

WAL retention can grow → monitor pg_wal size.

Primary keeps working even if standby is down.

Synchronous = Maximum Protection (Oracle) - Critical

WAL retention isn't the main concern, because transactions won't commit on primary if the standby is offline.

Disk usage won't grow indefinitely due to WALs for the offline standby, but the primary database pauses writes until the standby returns online.

7. Simple Analogy

Imagine the primary is a teacher writing on a blackboard.

The standbys are students copying the board in real-time.

As the teacher writes (WAL logs), students continuously copy (replay WAL) so everyone stays synchronized.

Some students may write slightly slower (async), or the teacher may wait for all students to copy before continuing (sync).

On Master:

1. Create Replication User (Primary – rac1)

```
# The replication user can stream WAL and connect as standby.
postgres=# \du
                                List of roles
Role name |                               Attributes
-----+-----
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS

postgres=#
postgres=# create user repuser with replication encrypted password
'repuser';
CREATE ROLE
postgres=#
postgres=# \du
                                List of roles
Role name |                               Attributes
-----+-----
postgres  | Superuser, Create role, Create DB, Replication, Bypass RLS
repuser   | Replication

postgres=#
```

2. Update postgresql.conf (Primary – rac1)

```
# Edit postgresql.conf

listen_addresses = '0.0.0.0' # ipv4 only
wal_level = 'replica'
hot_standby = on

archive_mode = on
archive_command = 'cp %p /pgArch/pgsql17/arch/%f'

max_wal_senders = 10
max_replication_slots = 10
wal_log_hints = on

# End
```

3. Update pg_hba.conf (Primary – rac1)

```
# replication privilege.
local   replication    all                                     trust
host    replication    all             127.0.0.1/32          trust
host    replication    all             ::1/128               trust

# Allow standby servers for replication
host    replication    repuser        192.168.2.22/32      scram-sha-256
host    replication    repuser        192.168.3.20/32      scram-sha-256
```

4. Restart Configuration (Primary – rac1)

```
[root@rac1 ~]# systemctl stop postgresql-17.service
[root@rac1 ~]# systemctl start postgresql-17.service
[root@rac1 ~]#

postgres=# show wal_level;
 wal_level
-----
 replica <---
(1 row)

postgres=# show hot_standby;
 hot_standby
-----
 on <----
```

```
(1 row)

postgres=# show listen_addresses;
 listen_addresses
-----
 0.0.0.0
(1 row)

postgres=#
```

On Standbys

5. Stop PostgreSQL on Standbys

5.1 Run from rac2

```
[root@rac2 ~]# systemctl stop postgresql-17.service
[root@rac2 ~]#
```

5.2 Run from rac3

```
[root@rac3 ~]# systemctl stop postgresql-17.service
[root@rac3 ~]#
```

6. Delete all files under data directory

6.1 Run from rac2

```
[postgres@rac2 ~]$ rm -rf /pgData/pgsql17/data/*
[postgres@rac2 ~]$ rm -rf /pgWal/pgsql17/wal/*
[postgres@rac2 ~]$ rm -rf /pgArch/pgsql17/arch/*
```

6.2 Run from rac3

```
[postgres@rac3 ~]$ rm -rf /pgData/pgsql17/data/*
[postgres@rac3 ~]$ rm -rf /pgWal/pgsql17/wal/*
```

```
[postgres@rac3 ~]$ rm -rf /pgArch/pgsql17/arch/*
```

7. Run the backup on the standby to copy data from the primary

7.1 Run from rac2

```
# Each standby must have a unique slot name
# Physical slots are only for streaming replication, not logical
replication.
# Oracle - ARCHIVELOG DELETION POLICY TO APPLIED ON STANDBY =
rac2_standby_slot
# Replication slots retain WAL files until apply on standby

[postgres@rac2 ~]$ pg_basebackup -h 192.168.2.21 -U repuser -p 5432 -D
/pgData/pgsql17/data -Fp -Xs -P -R -C -S rac2_standby_slot
Password:
24406/24406 kB (100%), 1/1 tablespace
[postgres@rac2 ~]$
```

7.2 Run from rac3

```
# Each standby must have a unique slot name
# Physical slots are only for streaming replication, not logical
replication.
# Oracle - ARCHIVELOG DELETION POLICY TO APPLIED ON STANDBY =
rac3_standby_slot
# Replication slots retain WAL files until apply on standby

[postgres@rac3 ~]$ pg_basebackup -h 192.168.2.21 -U repuser -p 5432 -D
/pgData/pgsql17/data -Fp -Xs -P -R -C -S rac3_standby_slot
Password:
24407/24407 kB (100%), 1/1 tablespace
[postgres@rac3 ~]$
```

8. Verify primary_conninfo on Standbys

8.1 Run from rac2

```

# This automatically creates physical replication slots on the primary
while taking the base backup.
# Check primary_conninfo in postgresql.auto.conf

[postgres@rac2 ~]$ ls -ltr /pgData/pgsql17/data/postgresql.auto.conf
-rw-----. 1 postgres postgres 452 Oct 17 07:56
/pgData/pgsql17/data/postgresql.auto.conf
[postgres@rac2 ~]$
[postgres@rac2 ~]$ ls -ltr /pgData/pgsql17/data/standby.signal
-rw-----. 1 postgres postgres 0 Oct 17 07:56
/pgData/pgsql17/data/standby.signal
[postgres@rac2 ~]$

[postgres@rac2 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.21 port=5432 sslmode=prefer sslnegotiation=postgres
sslcompression=0 sslcertmode=allow sslni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'rac2_standby_slot'
[postgres@rac2 ~]$

```

8.2 Run from rac3

```

# This automatically creates physical replication slots on the primary
while taking the base backup.
# Check primary_conninfo in postgresql.auto.conf

[postgres@rac3 ~]$ ls -ltr /pgData/pgsql17/data/postgresql.auto.conf
-rw-----. 1 postgres postgres 452 Oct 17 08:00
/pgData/pgsql17/data/postgresql.auto.conf
[postgres@rac3 ~]$
[postgres@rac3 ~]$ ls -ltr /pgData/pgsql17/data/standby.signal
-rw-----. 1 postgres postgres 0 Oct 17 08:00
/pgData/pgsql17/data/standby.signal
[postgres@rac3 ~]$
[postgres@rac3 ~]$ cat /pgData/pgsql17/data/postgresql.auto.conf
# Do not edit this file manually!
# It will be overwritten by the ALTER SYSTEM command.
primary_conninfo = 'user=repuser password=repuser channel_binding=prefer
host=192.168.2.21 port=5432 sslmode=prefer sslnegotiation=postgres

```



```
sslcompression=0 sslcertmode=allow sslsni=1
ssl_min_protocol_version=TLSv1.2 gssencmode=prefer krbsrvname=postgres
gssdelegation=0 target_session_attrs=any load_balance_hosts=disable'
primary_slot_name = 'rac3_standby_slot'
[postgres@rac3 ~]$
```

9. Start PostgreSQL on Standbys

9.1 Run from rac2

```
[root@rac2 ~]# systemctl start postgresql-17.service
[root@rac2 ~]#

[postgres@rac2 ~]$ cat /pgData/pgsql17/data/log/postgresql-Fri.log
..
..
2025-10-17 08:08:43.168 EDT [9064] LOG:  starting PostgreSQL 17.6 on
x86_64-pc-linux-gnu, compiled by gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-
26), 64-bit
2025-10-17 08:08:43.169 EDT [9064] LOG:  listening on IPv4 address
"0.0.0.0", port 5432
2025-10-17 08:08:43.175 EDT [9064] LOG:  listening on Unix socket
"/run/postgresql/.s.PGSQL.5432"
2025-10-17 08:08:43.187 EDT [9064] LOG:  listening on Unix socket
"/tmp/.s.PGSQL.5432"
2025-10-17 08:08:43.198 EDT [9070] LOG:  database system was interrupted;
last known up at 2025-10-17 07:56:39 EDT
2025-10-17 08:08:44.673 EDT [9070] LOG:  starting backup recovery with redo
LSN 0/15000028, checkpoint LSN 0/15000080, on timeline ID 1
2025-10-17 08:08:44.673 EDT [9070] LOG:  entering standby mode
2025-10-17 08:08:44.684 EDT [9070] LOG:  redo starts at 0/15000028
2025-10-17 08:08:44.689 EDT [9070] LOG:  completed backup recovery with
redo LSN 0/15000028 and end LSN 0/15000120
2025-10-17 08:08:44.689 EDT [9070] LOG:  consistent recovery state reached
at 0/15000120
2025-10-17 08:08:44.689 EDT [9064] LOG:  database system is ready to accept
read-only connections
2025-10-17 08:08:44.760 EDT [9071] LOG:  started streaming WAL from primary
at 0/16000000 on timeline 1
[postgres@rac2 ~]$
```

9.2 Run from rac3

```

[root@rac3 ~]# systemctl start postgresql-17.service
[root@rac3 ~]#

[postgres@rac3 ~]$ cat /pgData/pgsql17/data/log/postgresql-Fri.log
..
..
2025-10-17 08:08:53.603 EDT [8683] LOG:  starting PostgreSQL 17.6 on
x86_64-pc-linux-gnu, compiled by gcc (GCC) 8.5.0 20210514 (Red Hat 8.5.0-
26), 64-bit
2025-10-17 08:08:53.604 EDT [8683] LOG:  listening on IPv4 address
"0.0.0.0", port 5432
2025-10-17 08:08:53.612 EDT [8683] LOG:  listening on Unix socket
"/run/postgresql/.s.PGSQL.5432"
2025-10-17 08:08:53.623 EDT [8683] LOG:  listening on Unix socket
"/tmp/.s.PGSQL.5432"
2025-10-17 08:08:53.634 EDT [8688] LOG:  database system was interrupted;
last known up at 2025-10-17 08:00:22 EDT
2025-10-17 08:08:55.128 EDT [8688] LOG:  starting backup recovery with redo
LSN 0/17000028, checkpoint LSN 0/17000080, on timeline ID 1
2025-10-17 08:08:55.128 EDT [8688] LOG:  entering standby mode
2025-10-17 08:08:55.139 EDT [8688] LOG:  redo starts at 0/17000028
2025-10-17 08:08:55.144 EDT [8688] LOG:  completed backup recovery with
redo LSN 0/17000028 and end LSN 0/17000158
2025-10-17 08:08:55.144 EDT [8688] LOG:  consistent recovery state reached
at 0/17000158
2025-10-17 08:08:55.144 EDT [8683] LOG:  database system is ready to accept
read-only connections
2025-10-17 08:08:55.206 EDT [8689] LOG:  started streaming WAL from primary
at 0/18000000 on timeline 1
[postgres@rac3 ~]$

```

10. Verify Streaming Replication

10.1 Run from rac1

```

[postgres@rac1 ~]$ psql -U postgres -c "SELECT pid, client_addr, state,
sync_state FROM pg_stat_replication;"
  pid  | client_addr  | state   | sync_state
-----+-----+-----+-----
 12282 | 192.168.2.22 | streaming | async
 12283 | 192.168.3.20 | streaming | async
(2 rows)

```

```
[postgres@rac1 ~]$
```

```
[postgres@rac1 ~]$ psql -U postgres -c "SELECT pg_is_in_recovery();"  
pg_is_in_recovery
```

```
-----  
f <---- it means primary  
(1 row)
```

```
[postgres@rac1 ~]$
```

```
postgres=# SELECT * FROM pg_replication_slots;
```

```
 slot_name      | plugin | slot_type | datoid | database | temporary | active | active_pid |  
xmin | catalog_xmin | restart_lsn | confirmed_flush_lsn | wal_status | safe_wal_size | two_phase |  
inactive_since |  
conflicting | invalidation_reason | failover | synced
```

```
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----  
---+-----+-----+-----+-----+-----+-----+-----+-----+-----  
-----+  
-----+-----+-----+-----+-----+-----+-----+-----+-----
```

```
rac2_standby_slot |      | physical |      |      | f      | t      | 12282 |  
|          | 0/18023510 |      |      | reserved |      | f      |  
|
```

```
          |      | f      | f
```

```
rac3_standby_slot |      | physical |      |      | f      | t      | 12283 |  
|          | 0/18023510 |      |      | reserved |      | f      |  
|
```

```
          |      | f      | f
```

```
(2 rows)
```

```
postgres=#
```

```
postgres=# CREATE TABLE emp (name TEXT, designation TEXT, project TEXT,  
company TEXT);
```

```
CREATE TABLE
```

```
postgres=# INSERT INTO emp VALUES ('Sugi', 'DBA', 'Jetstar', 'iGATE');  
INSERT 0 1
```

```
postgres=# INSERT INTO emp VALUES ('Teja', 'DBA', 'RCM', 'iGATE');  
INSERT 0 1
```

```
postgres=# INSERT INTO emp VALUES ('RAJ', 'DBA', 'RCM', 'iGATE');  
INSERT 0 1
```

```
postgres=#
```

```
postgres=# SELECT * FROM EMP;
```

```
 name | designation | project | company
```

```
-----+-----+-----+-----
```

```
Sugi | DBA          | Jetstar | iGATE
```

Teja	DBA	RCM	iGATE
RAJ	DBA	RCM	iGATE

(3 rows)

postgres=#

10.2 Run from rac2

```
[postgres@rac2 ~]$ psql -U postgres -c "SELECT pg_is_in_recovery();"
pg_is_in_recovery
-----
t <---- it means standby
(1 row)
```

```
[postgres@rac2 ~]$
[postgres@rac2 ~]$ psql
psql (17.6)
Type "help" for help.
```

```
postgres=# SELECT * FROM EMP;
 name | designation | project | company
-----+-----+-----+-----
 Sugi | DBA         | Jetstar | iGATE
 Teja | DBA         | RCM     | iGATE
 RAJ  | DBA         | RCM     | iGATE
(3 rows)
```

postgres=#

```
2025-10-17 08:13:43.297 EDT [9068] LOG:  restartpoint starting: time
2025-10-17 08:13:43.333 EDT [9068] LOG:  restartpoint complete: wrote 1
buffers (0.0%); 0 WAL file(s) added, 0 removed, 3 recycled; write=0.002 s,
sync=0.001 s, total=0.037 s; sync files=0, longest=0.000 s, average=0.000
s; distance=49152 kB, estimate=49152 kB; lsn=0/180000B8, redo
lsn=0/18000060
2025-10-17 08:13:43.333 EDT [9068] LOG:  recovery restart point at
0/18000060
[postgres@rac2 ~]$
```

10.3 Run from rac3

```
[postgres@rac3 ~]$ psql -U postgres -c "SELECT pg_is_in_recovery();"
pg_is_in_recovery
```

```
-----
t <---- it means standby
(1 row)
```

```
[postgres@rac3 ~]$
[postgres@rac3 ~]$ psql
psql (17.6)
Type "help" for help.
```

```
postgres=# SELECT * FROM EMP;
 name | designation | project | company
-----+-----+-----+-----
  Sugi | DBA         | Jetstar | iGATE
  Teja | DBA         | RCM     | iGATE
  RAJ  | DBA         | RCM     | iGATE
(3 rows)
```

```
postgres=#
```

```
2025-10-17 08:13:53.731 EDT [8686] LOG:  restartpoint starting: time
2025-10-17 08:13:53.757 EDT [8686] LOG:  restartpoint complete: wrote 1
buffers (0.0%); 0 WAL file(s) added, 0 removed, 1 recycled; write=0.002 s,
sync=0.001 s, total=0.026 s; sync files=0, longest=0.000 s, average=0.000
s; distance=16384 kB, estimate=16384 kB; lsn=0/180000B8, redo
lsn=0/18000060
2025-10-17 08:13:53.757 EDT [8686] LOG:  recovery restart point at
0/18000060
[postgres@rac3 ~]$
```

11 Verify lag from rac1 (Primary)

```
[postgres@rac1 ~]$ psql -U postgres -c "
> SELECT
> application_name AS standby_name,
> client_addr AS standby_ip,
```

```

> state,
> sync_state,
> pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn) AS byte_lag
> FROM pg_stat_replication;
> "
standby_name | standby_ip | state | sync_state | byte_lag
-----+-----+-----+-----+-----
walreceiver | 192.168.2.22 | streaming | async | 0
walreceiver | 192.168.3.20 | streaming | async | 0
(2 rows)

[postgres@rac1 ~]$

```

Caution: Your use of any information or materials on this website is entirely at your own risk. It is provided for educational purposes only. It has been tested internally, however, we do not guarantee that it will work for you. Ensure that you run it in your test environment before using.

Thank you,

Rajasekhar Amudala

Email: br8dba@gmail.com

Linkedin: <https://www.linkedin.com/in/rajasekhar-amudala/>